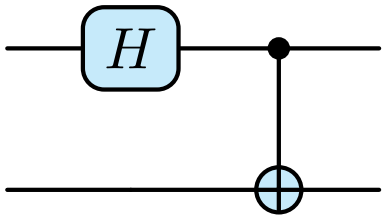
Quantum Information Processing

Lecture Notes

Instructor: Prof. Chiranjib Mitra



Sagnik Seth

Dept of Physical Sciences
IISER Kolkata

Contents

Lecture 00: Introduction

The course pertains to the topics in quantum information and how to process this information. Quantum computing has recently garnered a huge hype in both academia and industries and the potential applications are being studied. We will deal with the basic introduction to some aspects of quantum information theory in this course.

Although plethora of literature is available for the subject, the references for the course will be:

- **Quantum Computation and Quantum Information**; *Michael Nielsen, Isaac Chuang.* Termed as the 'bible' of quantum information.
- **Preskill Notes**; *John Preskill.* The link to the notes page is `www.preskill.caltech.edu/ph229/`
- **The Physics of Quantum Information: Quantum Cryptography, Quantum Teleportation, Quantum Computation**; *Bouwmeester, Ekert, Zeilinger*

The grading for the course consists of 20% from the midsem, 20% from two best out of three class tests, 30% from the endsem, 10% from class participation and rest 20% is contributed either from assignments or class notes or both.

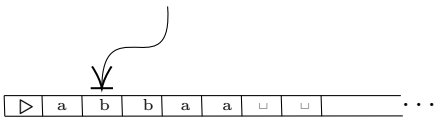
Lecture 01: Historical Development

Let us begin with an overview into what led to the development of quantum computation. Quantum computation involves the fields of physics and theoretical computer science. Around 1930s, *Alan Turing* developed an abstract notion of a programmable computer, the *Turing machine.*

The motivation behind the Turing machine was how humans performed calculations. When we compute or calculate something, we see that problem, formalise the solution in our mind and write it down. In a similar way, the Turing machine consists of a *tape* with some alphabets and a *tape head.* The tape head operates on the alphabet directly below it. This acts as the *state* for the machine and through different transition functions, the machine runs until it encounters a *accept* or a *reject state*, after which it halts.

Definition 1 (Turing Machine):
A Turing Machine M is a 6-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, H)$ where,

- Q is a finite set of states.
- Σ is a finite set of input alphabets that are on the tape. Σ does not include special characters such as $\triangleright, \leftarrow, \rightarrow, \sqcup$.
- $\Gamma = \Sigma \cup \{\triangleright, \sqcup\}$ is a finite set of tape alphabets.
- $\delta : (Q\ H) \times \Gamma \to Q \times \{\Gamma \cup \{\leftarrow, \rightarrow\}\}$ is the transition function. It takes a state and returns another state and a tape alphabet or the next direction in which to move.
- q_0 is the start state.
- $H = \{q_{\text{acc}}, q_{\text{rej}}\} \subset Q$ is the set of halting states, that is, if any of these states are returned by the transition function, the machine stops.



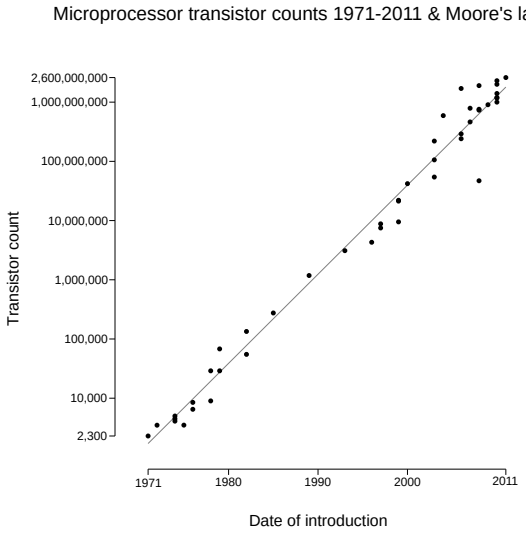
The above diagram shows a tape (suppose at state q) and the tape head with an arrow reading the letter b. It will return a new state q_1 and a symbol either $\leftarrow$ or $\rightarrow$, according to which the tape moves left or

right. If somehow the tape reaches the left end, it encounters ▷, which will force the tape to move right only. In this way, the machine goes on, unless it returns one of the halting states.

The concept of a *Universal Turing Machine* (UTM) was also formalised, which can simulate any other Turing machine to solve a problem. It completely captures what it means to perform a task by an algorithmic process.

This led to the following thesis, that, every physically realisable computation device can be simulated by a Turing machine. This was formulated by Turing and Alonzo Church (who introduced λ-calculus which is equivalent to a Turing machine) and is thus called the *Church-Turing thesis*.

Around the same time, rapid developments were being made in the hardware industry. Around 1940s, Bardeen, Brattain, and Shockley of the Bell Labs discovered the transistor which lead to development of electronic chips. As technology advanced, more and more transistors were accomodated on a chip. This led to an empiricial observation by Gordon Moore, that *computing power roughly doubles once every two years at a constant cost*.



This is problematic, since there will be a time when the distance between the transistors will be entering the quantum regime (as the chip size remains constant) and classical physics breaks down as quantum effects begin to interfere in the functioning of electronic devices as they are made smaller and smaller.

The solution for this was to change the paradigm. Instead of treating quantum effects as a interference, we started to develop quantum computers which uses quantum mechanics as an operating system to perform computations.

Note that, a classical computer *can* simulates quantum mechanics, but not efficiently! Classical algorithms would take *superpolynomial* or *exponential* time to run a quantum computation.

This concept of efficiency led to the strengthening of the Church-Turing thesis

"Any algorithmic process can be *efficiently* simulated by a universal Turing machine"

This is helpful since, if this thesis is true, then instead of analysing different models of computation for a given task, we can restrict ourselves to the Turing machine model for the computation of the task.

A challenge to this statement came when *randomised algorithms* were introduced. These algorithms used random numbers to perform a task. However, Turing machines were *deterministic* and thus suggesting that efciently soluble problems exist which cannot be efciently solved on a deterministic Turing machine. One such algorithm is the *Solovay-Strassen* test which is a primality test.

Algorithm 1 (Solovay-Strassen):

Require: n, a value to test for primality and k, a parameter that determines the accuracy of the test
Ensure: composite if n is composite and otherwise prime with some *uncertainty*

```
for n = 1, ..., k do
    choose a randomly from [2, n − 1]
    x ← (a/n)
    if x = 0 or a^((n−1)/2) ≢ x (mod n) then
        return composite
    end if
end for
return probably prime
```

The problem posed by probabilistic algorithms can be resolved by the modified Strong Church-Turing thesis,

> "Any algorithmic process can be simulated efciently using a probabilistic Turing machine"

This modification is kinda *ad hoc* and leads us to think that someday some new kind of computation will be developed which will lead to some further modification in the thesis. Thus we need to find a single model of computation which is guaranteed to simulate any other model of computation.

On this note, David Deutsch tried to use physical laws to develop a computational model. He attempted to formalise a model which will simulate any arbitrary physical system. Since quantum mechanics makes the foundation of natural laws, the idea of a *universal quantum computer* was established!

In his 1981 paper *Simulating Physics with Computer*, Feynman proposed the notion of a quantum computer. In a quantum computer, the algorithms also need to be quantum mechanical.

In 1994, Peter Shor developed an algorithm that enables finding prime factors of a number. This problem is in general hard, since given a number, say, 2153415191 we have to grind our minds to find out that it has two prime factors 64817 and 33223. However, the reverse problem is easy, given a number p we can check whether it is a factor of n, just by simple division.

The most efficient classical algorithm to factor large integers is the **general number field sieve** which has a sub-exponential time complexity, of the order of, $\exp\left(\left(\frac{64}{9}\right)^{1/3}(\ln N)^{1/3}(\ln\ln N)^{2/3}\right)$. However, as the number of digits increases, this takes a lot of time to factor. For about 400 digits, it takes 10^{10} years to factor, which is about the age of the Universe. On the other hand, on a quantum computer, using Shor's algorithm, it would take around three years to factorise a 400 digit number, which is way way better in time.

Lecture 02: Qubits

Classical computers uses classical bits 0 and 1 which represents the different modes of a transistor. However, for quantum computers, the fundamental unit of computation needs to be modified. We thus use *quantum bits* or *qubits* as the basis of quantum computation.

Suppose we have a two dimensional Hilbert space $\mathcal{H}$, spanned by the vectors $\{|0\rangle, |1\rangle\}$, which are the eigenstates of σ_z. Any arbitrary state in the Hilbert space can be written as a superposition of these vectors,

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \qquad \alpha, \beta \in \mathbb{C}, |\alpha|^2 + |\beta|^2 = 1$$

where the last condition used for normalisation of the vector. The states $|0\rangle$ and $|1\rangle$ are orthonormal, that is, $\langle 0|0\rangle = \langle 1|1\rangle = 1$ and $\langle 0|1\rangle = 0$. In the computational (standard) basis,

$$|0\rangle = \begin{pmatrix}1\\0\end{pmatrix} \quad |1\rangle = \begin{pmatrix}0\\1\end{pmatrix} \quad \Longrightarrow \quad |\psi\rangle = \begin{pmatrix}\alpha\\\beta\end{pmatrix} \quad \langle\psi| = \begin{pmatrix}\alpha^* & \beta^*\end{pmatrix} \tag{1}$$

Qubits are thus two level systems which are used for computation. These can be visualised as the ground and first excited state of an atom. Different systems are being used as qubits to harness the power of quantum computation, such as:

- Electron spins $|\uparrow\rangle$ and $|\downarrow\rangle$
- Nuclear spin
- Two-level atomic systems
- Cooper pair in a box (superconducting)
- Ion-trapped qubits
- Cavity QED
- Quantum dots
- Dopants in silicon
- NV centres in diamond
- Photon polarisation

These different systems have their own pros and cons based on *scalability*, *decoherence*, etc. Computations cannot be done by a single qubit, hence we need to introduce systems with more than two qubits. Let us consider two qubits, which is equivalent to combining two spin-half systems. These two spin-half objects live in their separate Hilbert spaces, say $\mathcal{H}_\mathrm{A}$ and $\mathcal{H}_\mathrm{B}$.

The direct product states are just $|\uparrow\uparrow\rangle, |\uparrow\downarrow\rangle, |\downarrow\uparrow\rangle$ and $|\downarrow\downarrow\rangle$. $|ab\rangle$ is the short-hand notation for the tensor product $|a\rangle \otimes |b\rangle$, where $|a\rangle \in \mathcal{H}_\mathrm{A}$ and $|b\rangle \in \mathcal{H}_\mathrm{b}$. Let us consider the direct sum states too.

From the addition of angular momentum, we know that the total spin operator is given by $\mathbf{S} = \mathbf{S}_1 + \mathbf{S}_2$ where $\mathbf{S}_1$ and $\mathbf{S}_2$ are the spin operators of the individual spins. We now want to find the states of the type $|S, S_z\rangle$. Note that, $s_1 = s_2 = \frac{1}{2}$ and hence $S = s_1 + s_2, s_1 + s_2 - 1 \ldots |s_1 - s_2| \implies S = 0, 1$.

If $S = 0$ then we have $|0, 0\rangle = \frac{1}{\sqrt{2}}(|\uparrow\downarrow\rangle - |\downarrow\uparrow\rangle)$ which is the singlet state. If $S = 1$, then we have the three triplet states $|11\rangle = |\uparrow\uparrow\rangle, |10\rangle = \frac{1}{\sqrt{2}}(|\uparrow\downarrow\rangle + |\downarrow\uparrow\rangle), |1, -1\rangle = |\downarrow\downarrow\rangle$. We will mostly use the direct product states, identifying $|\uparrow\rangle$ with $|0\rangle$ and $|\downarrow\rangle$ with $|1\rangle$, which leads to the basis of the two qubit states to be,

$$|00\rangle \qquad |01\rangle \qquad |10\rangle \qquad |11\rangle$$

Let us now see what these states will be in the computational basis. For example, using Eq. (1), we can write $|00\rangle$ in the following way,

$$|00\rangle = \begin{pmatrix}1\\0\end{pmatrix} \otimes \begin{pmatrix}1\\0\end{pmatrix} = \begin{bmatrix}1\begin{pmatrix}1\\0\end{pmatrix}\\ 0\begin{pmatrix}1\\0\end{pmatrix}\end{bmatrix} = \begin{pmatrix}1\\0\\0\\0\end{pmatrix}$$

It is easy to see that $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$ forms an orthonormal basis of the four dimensional complex vector space, $\mathbb{C}^4$. In general, a n-qubit system is 2^n dimensional and any arbitrary state can be expressed as,

$$\psi = \sum_{i=0}^{2^n-1} a_i |x_i\rangle \qquad \sum_i |a_i|^2 = 1 \qquad a_i \in \mathbb{C}$$

where x_i is the n-digit *binary representation* of the number i. For example, if we take 3 qubits, then $x_0 = 000, x_1 = 001, x_2 = 010, x_3 = 011, x_4 = 100, x_5 = 101, x_6 = 110, x_7 = 111$.

Lecture 03: Gates

Having seen the basic building blocks for quantum computation, let us see how the computation can be actually carried out. A quantum computer is expected to necessarily satisfy the following conditions put forth by *David P. DiVincenzo* in 1996:

- A scalable system with well-characterised qubits, that is, the system should be able to accomodate enough number of qubits necessary for computation.

- Initialising the qubits to some simple *fiducial state*, so that computations can be carried forward from there.
- Long relevant *coherence time*, that is, the system does not undergo decoherence until computation is finished.
- A universal set of quantum gates, which will carry out the computation by evolving the states. These gates need to be *unitary* as required by probability conservation.
- A specific measurement capability, that is, there should be provision to read out the results after computation is done.

Classical computations are carried out by combination of different logic gates which are based on Boolean algebra.

Boolean Algebra deals with logical operations of binary variables. A Boolean function f is generally defined as $f : \{0,1\}^k \to \{0,1\}$, that is, it takes input from k cartesian products of the set $\{0,1\}$ and returns a single output which can be 0 or 1. The basic boolean functions are:

- **NOT Gate**

  It is a unary operator which takes a single input and returns the negation of the input. It is represented by the symbol $\overline{\text{A}}$. If input is 1, it return 0 and if input is 0, it returns 1.

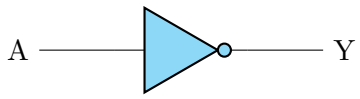


Figure 1: Representation of NOT Gate

- **AND Gate**

  It is a binary operator which takes two inputs and returns the logical AND of the inputs. It is represented by the symbol $\text{A} \cdot \text{B}$. It returns 1 only if both inputs are 1, otherwise it returns 0.

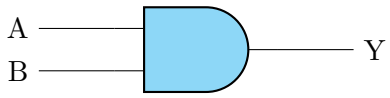


Figure 2: Representation of AND Gate

- **OR Gate**

  It is a binary operator which takes two inputs and returns the logical OR of the inputs. It is represented by the symbol $\text{A} + \text{B}$. It returns 1 if any of the inputs is 1, otherwise it returns 0.

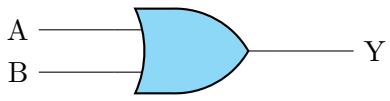


Figure 3: Representation of OR Gate

- **XOR Gate**

  It is a binary operator which takes two inputs and returns the logical XOR of the inputs. It is represented by the symbol $\text{A} \oplus \text{B}$. It can be showed that the expression for XOR is equivalent to $\overline{\text{A}}\text{B} + \text{A}\overline{\text{B}}$. It returns 1 if the inputs are different, otherwise it returns 0.

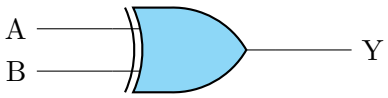


Figure 4: Representation of XOR Gate

- **NAND Gate**

  It is a binary operator which takes two inputs and returns the logical NAND of the inputs. It is represented by the symbol $\overline{\mathrm{AB}}$. Thus, it is a composition of AND and NOT operations. From de Morgan's law, it can be shown that NAND is equivalent to $\overline{\mathrm{A}} + \overline{\mathrm{B}}$.

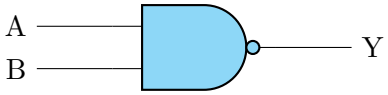


Figure 5: Representation of NAND Gate

- **NOR Gate** It is a binary operator which takes two inputs and returns the logical NOR of the inputs. It is represented by the symbol $\overline{\mathrm{A} + \mathrm{B}}$. Thus, it is a composition of OR and NOT operations. From de Morgan's law, it can be shown that NOR is equivalent to $\overline{\mathrm{A}} \cdot \overline{\mathrm{B}}$.

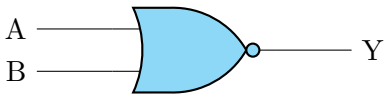


Figure 6: Representation of NOR Gate

Note that the except for the NOT gate, the number of output in each gate is always lesser than the number of input. This reduction of the computational *phase space complexity* renders these classical gates irreversible, since we cannot reconstruct the input given a particular output. This is problematic since we require quantum logic gates to be reversible due to unitarity condition, so a different approach is to be taken.

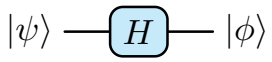
NOTE: Not all classical gates are irreversible. For example, the Toffoli gate is a universal reversible gate. It is three-input three-output controlled gate which means that the third input ('target') is controlled by the first two inputs ('control'), both of which do not change. That is, the target bit will be inverted if the first and second bits are both 1. Given inputs a, b, c then the output is accordingly given as $y = c \oplus (a \cdot b)$.

Table 1: Truth Table for the Toffoli gate

A	B	C	Output (a,b,y)
0	0	0	0,0,0
0	0	1	0,0,1
0	1	0	0,1,0
0	1	1	0,1,1
1	0	0	1,0,0
1	0	1	1,0,1
1	1	0	1,1,1
1	1	1	1,1,0

There are many different quantum gates, as we will see[1]. One of the most useful gate is the single-qubit *Hadamard gate* represented as,

[1] The title page itself shows two such gates.



The Hadamard gate is a unitary gate and is defined by its action on the basis elements:

$$|0\rangle \xmapsto{H} |+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \qquad |1\rangle \xmapsto{H} |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

Note that $H|+\rangle = \frac{1}{\sqrt{2}}(H|0\rangle + H|1\rangle) = \frac{1}{\sqrt{2}}(|+\rangle + |-\rangle) = |0\rangle$. Similarly, $H|-\rangle = |1\rangle$. Thus, $H^2 = \mathbb{1}$ which implies that $H = H^{-1}$. This is also evident from the Hadamard matrix written using the computational basis[1],

$$H \equiv \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

3.1 Deutsch's Algorithm

Let us discuss a quantum algorithm which demonstrates 'quantum parallelism'. Parallelism is meant to convey the sense that multiple computations can be simultaneously performed which is not possible in classical computation.

The basic task is to determine whether a binary function $f : \{0,1\} \to \{0,1\}$ is *constant* or *balanced*. Constant means that $f(0) = f(1)$ while balanced means $f(0) \neq f(1)$. If we use classical computation, we have to make two evaluations of the function to determine whether it is balanced or constant. Let us see whether we can obtain any advantage if we use quantum computation.

For carrying out the task, we will use an *oracle* which is like a *black box*, where we can pass some queries and it returns us some answers. We use an oracle U_f which takes two states, $|x\rangle$ and $|y\rangle$ and returns two states $|x\rangle$ and $|y \oplus f(x)\rangle$. The basic algorithm is as follows:

- Start with a initial two qubit state $|0\rangle \otimes |1\rangle$
- Apply Hadamard gate on both qubits
- Apply U_f to the product state
- Apply Hadamard gate to the first qubit and measure it.

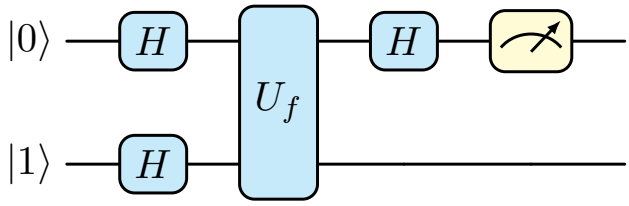


Figure 7: Circuit diagram for Deutsch's algorithm

We now elaborate each step of the algorithm. In the first step, we apply Hadamard gates on both of the initial qubits.

$$|\psi_1\rangle = (H \otimes H)|0\rangle \otimes |1\rangle = (H|0\rangle) \otimes (H|1\rangle) = |+\rangle \otimes |-\rangle \longrightarrow \frac{|0\rangle}{\sqrt{2}}\left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right) + \frac{|1\rangle}{\sqrt{2}}\left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right)$$

[1] To obtain the matrix element in a particular basis, we have $H_{ij} = \langle i|H|j\rangle$. So let's say H_{00} for the Hadamard gate will be,

$$\langle 0|H|0\rangle = \langle 0|+\rangle = \frac{1}{\sqrt{2}}(\langle 0|0\rangle + \langle 0|1\rangle) = \frac{1}{\sqrt{2}}$$

Now, note the following fact which we will use in the second step,

$$U_f \left|x\right\rangle \left|-\right\rangle = (-1)^{f(x)} \left|x\right\rangle \left|-\right\rangle$$

which follows because,

$$U_f\Big(\frac{\left|x\right\rangle \left|0\right\rangle - \left|x\right\rangle \left|1\right\rangle}{\sqrt{2}}\Big) = \frac{1}{\sqrt{2}}(\left|x\right\rangle \left|f(x)\oplus 0\right\rangle - \left|x\right\rangle \left|f(x)\oplus 1\right\rangle) = \frac{1}{\sqrt{2}}(\left|x\right\rangle \left|f(x)\right\rangle - \left|x\right\rangle \left|f(x)\oplus 1\right\rangle)$$

where we have used $a\oplus 0 = a$. Now, if $f(x) = 0$ then we have the state $\left|x\right\rangle \left(\frac{\left|0\right\rangle - \left|1\right\rangle}{\sqrt{2}}\right) = + \left|x\right\rangle \left|-\right\rangle$ while if $f(x) = 1$ we get the state $\left|x\right\rangle \left(\frac{\left|1\right\rangle - \left|0\right\rangle}{\sqrt{2}}\right) = - \left|x\right\rangle \left|-\right\rangle$. Hence in the second step, we have

$$\left|\psi_2\right\rangle = U_f \left|\psi_1\right\rangle = \frac{1}{\sqrt{2}}\Big((-1)^{f(0)} \left|0\right\rangle + (-1)^{f(1)} \left|1\right\rangle\Big) \left|-\right\rangle$$

If $f(0) = f(1)$ (balanced), then $\left|\psi_2\right\rangle = (-1)^{f(0)} \left|+\right\rangle \left|-\right\rangle$ and if $f(0) \neq f(1)$, then $(-1)^{f(0)}$ and $(-1)^{f(1)}$ will have opposite signs which will give us $\left|\psi_2\right\rangle = \pm \left|-\right\rangle \left|-\right\rangle$. The $\pm$ sign will depend on the value of the function, however, it is just an overall phase and will not matter to us. In the third step,we apply Hadamard gate to the first qubit,

$$(H\otimes \mathbb{1}) \left|\psi_2\right\rangle = \begin{cases} \left|0\right\rangle \left|-\right\rangle & f(0) = f(1) \\ \left|1\right\rangle \left|-\right\rangle & f(0) \neq f(1) \end{cases}$$

Hence by measuring the first qubit, if we obtain 0 then the function is constant, else the function is balanced.

Lecture 04: Classical Information